GIT QUEST | 30-DAY GIT LEARNING SERIES

Day 10: git commit --amend

Week 2: Undo and Branch | Level: Absolute Beginner

Created by Rutuja Naik

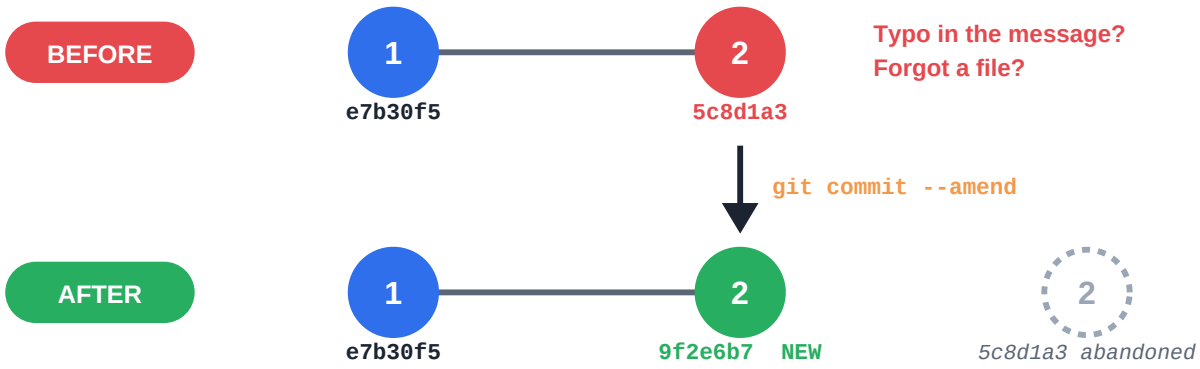
Today's mission

Double digits! You sealed a box and stuck on a label, then spotted a typo, or realised you forgot to pack one item. You do not need a new box: **reopen the last one**, fix it and seal it again.

- Fix the message of your last commit with `git commit --amend -m`
- Add a forgotten file with `--no-edit`
- Understand what amend really does under the hood
- Know the golden rule: never amend shared commits

1. The big idea: swap, not edit

`git commit --amend` replaces your **most recent commit**. Git does not edit the old commit: it builds a **new commit with a new hash** and abandons the old one. That is why `git log` shows a different ID afterwards.



Three ways to use it

You want to...	Command
Fix the message	**git commit --amend -m "Add menu"**
Add a forgotten file	**git add dessert.txt** **git commit --amend --no-edit**
Fix a small mistake in a file	**(edit the file)** **git add <file>** **git commit --amend --no-edit**

`--no-edit` means "keep the existing message".

2. Step by step

Step 1: Make a commit with a typo

```
$ git init -b main amend-lab
$ cd amend-lab
$ echo "Pasta" > menu.txt
$ git add menu.txt
$ git commit -m "Add mnu"
$ git log --oneline
a41c9d2 (HEAD -> main) Add mnu
```

Step 2: Fix the message

```
$ git commit --amend -m "Add menu"
$ git log --oneline
e7b30f5 (HEAD -> main) Add menu      <- new hash!
```

Step 3: Commit, then forget a file

```
$ echo "Dessert" >> menu.txt
$ git add menu.txt
$ git commit -m "Add dessert to menu"
$ echo "Dessert recipe" > dessert.txt
```

Step 4: Sneak the file into the same commit

```
$ git add dessert.txt
$ git commit --amend --no-edit
$ git show --stat
 menu.txt    | 1 +
 dessert.txt | 1 +
 2 files changed, 2 insertions(+)
```

Step 5: Check the history

```
$ git log --oneline
9f2e6b7 (HEAD -> main) Add dessert to menu
e7b30f5 Add menu
```

> **Golden rule**
> Only amend commits that are still **on your computer**. Once a commit has been pushed to GitHub (Week 3), other people may already have it. Rewriting it causes confusion. If a commit is already shared, make a new commit instead.

3. Good to know

Good to know	Details
Only the last commit	Amend works on the most recent commit. Older ones need tools you will meet later.
An editor opened?	Amend without -m or --no-edit opens an editor (often Vim). Press Esc, type :wq, press Enter to save and quit. Use :q! to quit without saving.
Amend vs reset --soft	git reset --soft HEAD~1 followed by git commit gives the same result in two steps. Amend is the shortcut.
Is the old commit gone?	Not instantly. git reflog (Day 26) can still find it.

4. Cheat sheet

Command	What it does
git commit --amend -m "msg"	Fix the last commit's message
git add <file>	Stage a forgotten file
git commit --amend --no-edit	Add staged changes, keep the message
git log --oneline	Notice the new hash
git show --stat	Which files are in the commit?

Your Day 10 commit
File: `days-1-to-10-summary.md`
Message: `docs(day10): add days 1-10 summary and amend notes`

5. Mermaid diagram GitHub draws for you

````
```mermaid
gitGraph
  commit id: "Add menu"
  commit id: "Add dessert to menu (amended)"
```
````

6. Challenge and quiz

Challenge: make a commit with a typo in the message and fix it with amend. Then forget a file and add it with `--no-edit`. Compare the hashes in `git log`.

Q1. Which flag keeps the old message when you amend?

Q2. Does amend edit the old commit or create a new one?

Q3. When should you never amend a commit?

Answers: 1) --no-edit. 2) A new commit with a new hash. 3) After the commit has been shared.

> **Coming up: Day 11**
> **What is a branch?** Meet one of the biggest ideas in Git.